Derivatives, Gradients, Jacobians & Hessians

The calculus behind optimization and backpropagation

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Why a calculus lecture?

Every step of training is a gradient step:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$$

So we must be fluent with

$$\frac{\mathrm{d}}{\mathrm{d}x}, \quad \frac{\partial}{\partial x_i}, \quad \nabla f, \quad J, \quad H.$$

These five objects are the entire vocabulary of optimization and backprop.

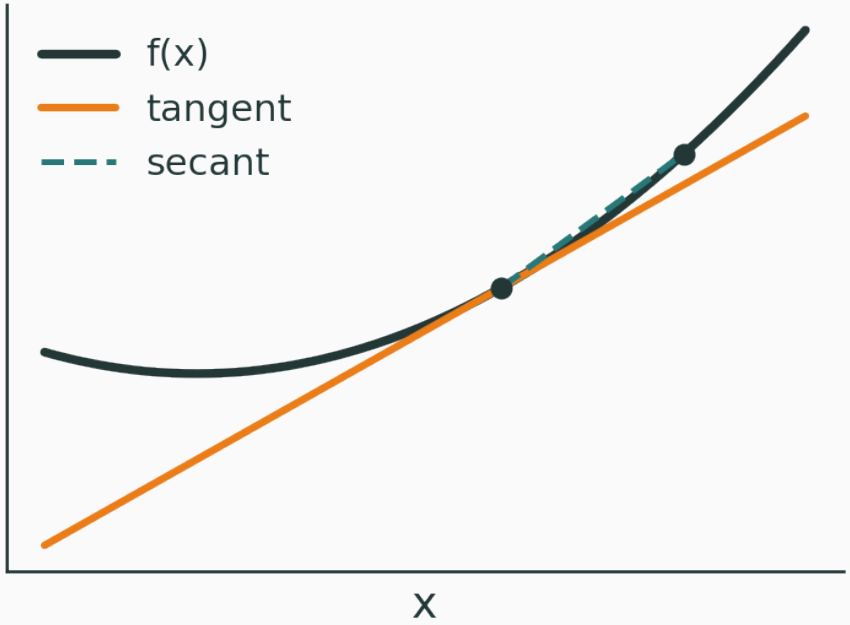
The big picture

Object	Function type	Tells us
derivative $f'(x)$	$\mathbb{R} \to \mathbb{R}$	slope
partial $\frac{\partial f}{\partial x_i}$	$\mathbb{R}^d \to \mathbb{R}$	sensitivity to one coordinate
gradient ∇f	$\mathbb{R}^d \to \mathbb{R}$	steepest-ascent direction
Jacobian J	$\mathbb{R}^n \to \mathbb{R}^m$	local linear map
Hessian H	$\mathbb{R}^d \to \mathbb{R}$	curvature

The scalar derivative

Derivative as slope

$$f'(x) = \frac{\mathrm{d}f}{\mathrm{d}x} = \lim_{\varepsilon \to 0} \frac{f(x+\varepsilon) - f(x)}{\varepsilon}$$



the secant slope → the tangent slope as $\varepsilon \to 0$

The derivative is a local linear model

D

Near a point x:

$$f(x + \Delta x) \approx f(x) + f'(x)\,\Delta x$$

derivative = best local linear approximation

Example: $f(x) = x^2$, $f'(3) = 6$, so $f(3 + \Delta x) \approx 9 + 6\,\Delta x$.

Interactive: tangent line I

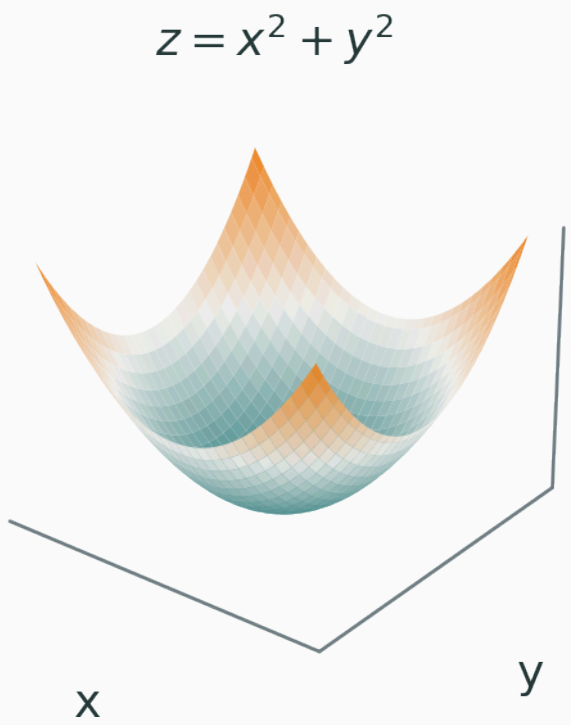
INTERACTIVE

(to build) — drag the point and Δx; watch the tangent, the secant, and the approximation error. Functions: x^2, $\sin x$, e^x, $\log(1 + e^x)$.

Q. When is the linear approximation good, and when does it break down?

Partial derivatives and surfaces

A function of two inputs is a **surface**: $f(x, y) = x^2 + y^2$.



input $(x, y) \rightarrow$ height $z = f(x, y)$

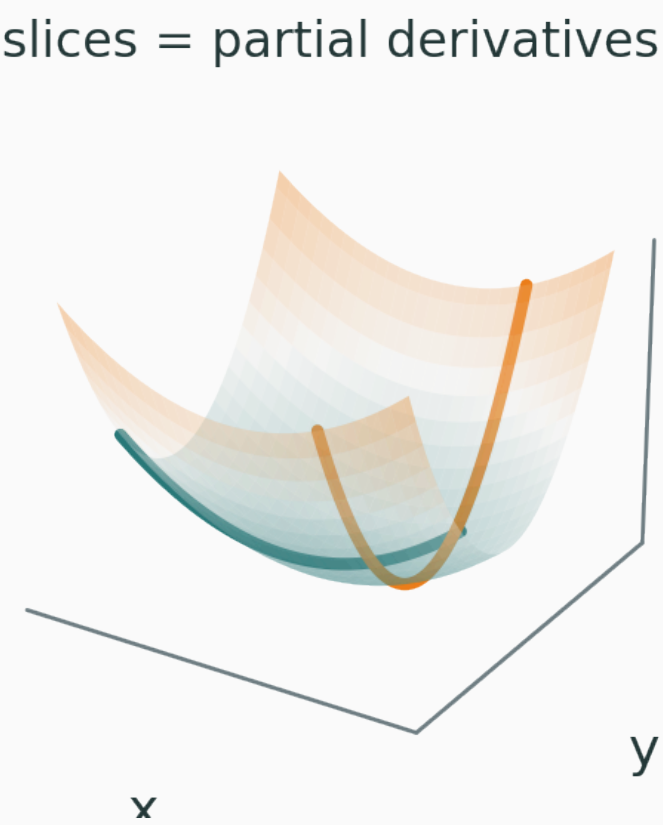
Partial derivatives

$\frac{\partial f}{\partial x}$ — change x, **hold y fixed**.

$\frac{\partial f}{\partial y}$ — change y, hold x fixed. For $f(x, y) = x^2 + y^2$:

$$\frac{\partial f}{\partial x} = 2x, \qquad \frac{\partial f}{\partial y} = 2y$$

A partial is the slope of a slice



For $f = x^2 + 3y^2$ at $(1, 2)$: $\frac{\partial f}{\partial x} = 2$, $\frac{\partial f}{\partial y} = 12$ — the function climbs **faster** along y.

Interactive: surface slices

I

INTERACTIVE

(to build) — move a point on a 3-D surface and read off the two slice slopes.
Functions: $x^2 + y^2$, $x^2 + 3y^2$, $\sin x \cos y$.

The gradient

Gradient: stack the partials

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{pmatrix}$$

For $f(x, y) = x^2 + y^2$: $\nabla f = \begin{pmatrix} 2x \\ 2y \end{pmatrix}$.

Gradient = local linear model

D

$$f(x + \Delta x) \approx f(x) + \nabla f(x)^\top \Delta x$$

the vector version of $f(x + \Delta x) \approx f(x) + f'(x)\Delta x$

Directional derivative

For a **unit** direction u:

$$D_u f(x) = \nabla f(x)^\top u$$

The rate of change along u is just the **projection** of the gradient onto u.

The gradient points uphill

D

By Cauchy–Schwarz, over all unit u:

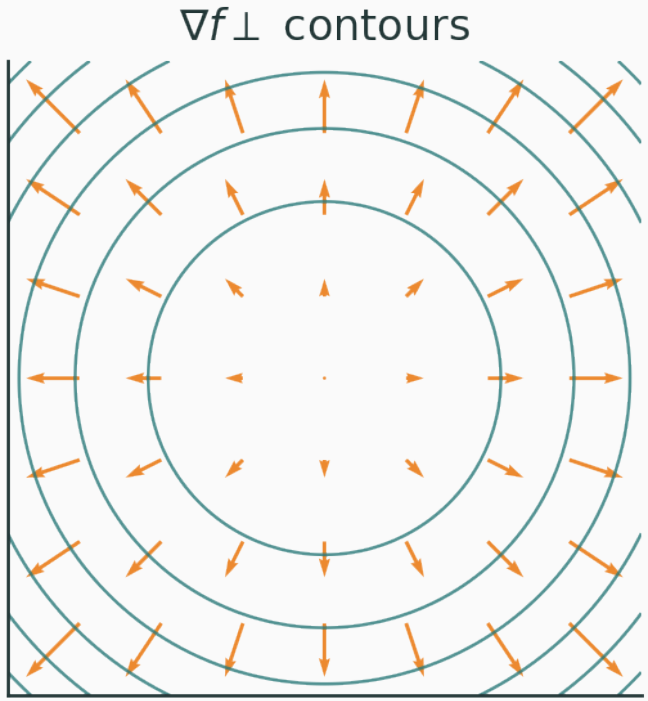
$$\nabla f(x)^\top u \le \|\nabla f(x)\|$$

with equality at $u = \nabla f(x)/\|\nabla f(x)\|$.

∇f = steepest ascent · $-\nabla f$ = steepest descent

The gradient is perpendicular to contours

A contour is the set $f(x, y) = c$. Along it, f does not change, so for any tangent v: $\nabla f^\top v = 0$.



Gradient descent on the contour map

$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

Each step moves **perpendicular** to the current contour, downhill. The learning rate η sets the step length.

Interactive: gradient field & descent

I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/optimizer-race

Watch gradient arrows on a contour map and a descent trajectory as you change η and the start point. Surfaces: $x^2 + y^2$, $x^2 + 10y^2$, $\sin x + \cos y$.

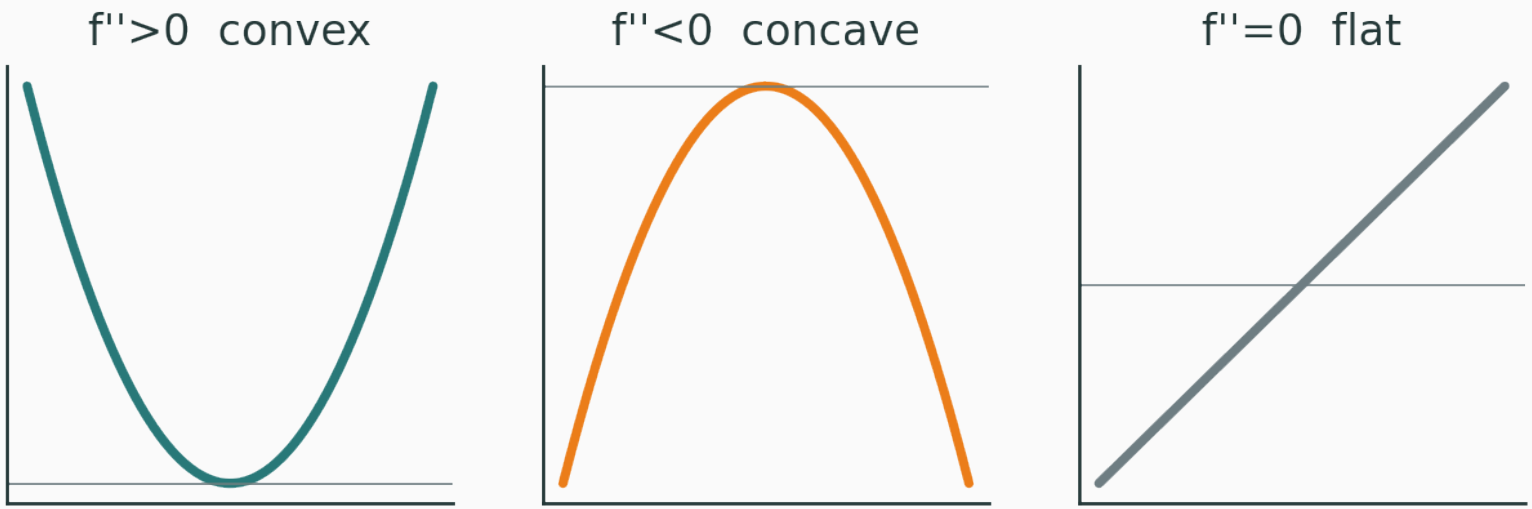
The Hessian and curvature

Second derivative in 1-D

$$f''(x) = \frac{\mathrm{d}^2 f}{\mathrm{d}x^2} \quad = \quad \text{how fast the slope changes}$$

$f(x) = x^2 \Rightarrow f'' = 2$ (bowl); $\quad f(x) = -x^2 \Rightarrow f'' = -2$ (hill).

Convex, concave, flat



The Hessian: matrix of second partials

$$H = \nabla^2 f, \quad H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

For two variables:

$$H = \begin{pmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{pmatrix}$$

Hessian example

D

$$f(x, y) = x^2 + 3y^2 \quad \Longrightarrow \quad \nabla f = \begin{pmatrix} 2x \\ 6y \end{pmatrix}, \quad H = \begin{pmatrix} 2 & 0 \\ 0 & 6 \end{pmatrix}$$

Curvature is **larger** in the y direction — the bowl is steeper that way.

Second-order local approximation

D

$$f(x+\Delta x) \approx f(x) + \underbrace{\nabla f(x)^\top \Delta x}_{\text{tilt (gradient)}} + \underbrace{\frac{1}{2}\Delta x^\top H(x) \Delta x}_{\text{curvature (Hessian)}}$$

gradient tilts · Hessian bends

Hessian geometry

optional

For a quadratic $f(x) = \frac{1}{2} x^\top A x$: $\nabla f = Ax, \quad H = A$.

- **eigenvectors** of H → the curvature **directions**;
- **eigenvalues** of H → the curvature **magnitudes**.

Classifying critical points

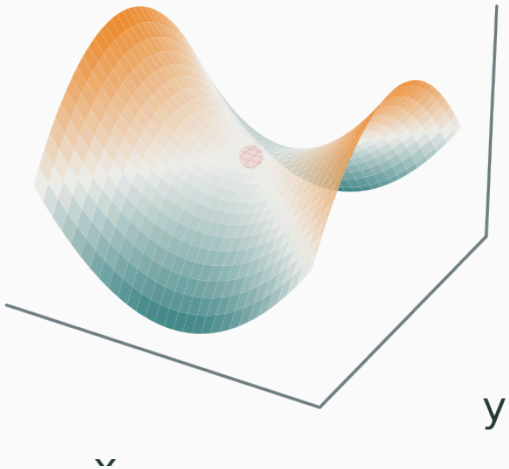
At a point where $\nabla f = 0$, the Hessian eigenvalues decide the shape:

Hessian eigenvalues	**Local shape**
all positive	local minimum
all negative	local maximum
mixed signs	saddle point
some zero	inconclusive / flat

A saddle point

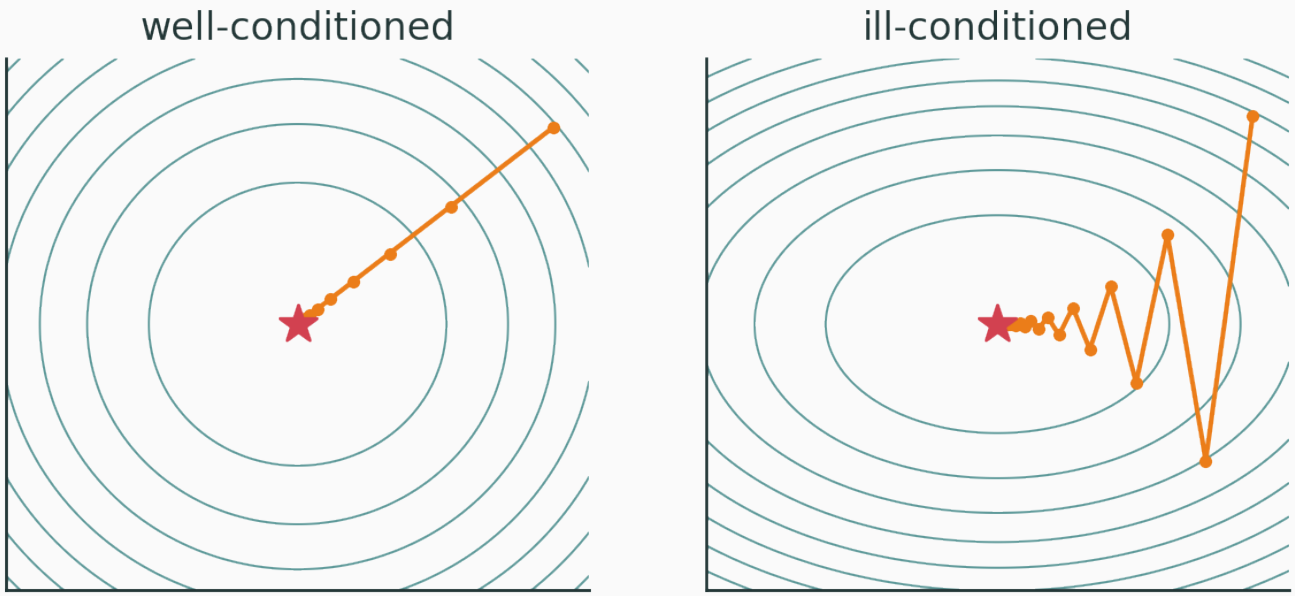
$$f(x, y) = x^2 - y^2, \quad \nabla f = \begin{pmatrix} 2x \\ -2y \end{pmatrix}, \quad H = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}$$

$f = x^2 - y^2$ (saddle)



$\nabla f = 0$ at the origin, but it is **not** a minimum

Why curvature matters for optimization



Elongated contours (a large **condition number**) make first-order gradient descent **zigzag** — much curvature one way, little the other.

Interactive: curvature & descent

I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/optimizer-race

Reshape a quadratic $ax^2 + by^2 + cxy$, see its Hessian eigenvectors, and watch the descent path zigzag.

Q. Why does the trajectory zigzag when the contours are stretched?

The Jacobian

When the gradient is not enough

The gradient is for **scalar** outputs, $f : \mathbb{R}^d \to \mathbb{R}$. But network layers map **vectors to vectors**, $f : \mathbb{R}^n \to \mathbb{R}^m$ — e.g. $z = Wx + b$. We need the **Jacobian**.

Jacobian: matrix of all partials

For $f : \mathbb{R}^n \to \mathbb{R}^m$:

$$J = \frac{\partial f}{\partial x} \in \mathbb{R}^{m \times n}, \qquad J_{ij} = \frac{\partial f_i}{\partial x_j}$$

$f(x + \Delta x) \approx f(x) + J(x)\,\Delta x$ **— a local linear map**

Two Jacobians you already know

D

Linear layer $f(x) = Wx + b$:

$$J = W$$

since $\frac{\partial z_i}{\partial x_j} = W_{ij}$.

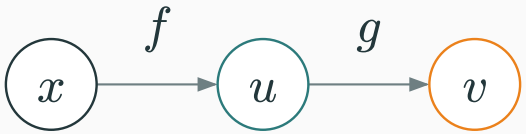
Elementwise $h = \varphi(z)$:

$$J = \text{diag}(\varphi'(z_1), ..., \varphi'(z_m))$$

(for ReLU, entries are 0 or 1).

The chain rule, with Jacobians

D



$$\frac{\partial v}{\partial x} = \frac{\partial v}{\partial u}\frac{\partial u}{\partial x} \quad \Longrightarrow \quad J_{v,x} = J_{v,u}\, J_{u,x}$$

Why we avoid full Jacobians

optional

A single layer with $x, h \in \mathbb{R}^{4096}$ has a Jacobian of $4096 \times 4096 \approx 1.7 \times 10^7$ entries.

Frameworks never materialize J. They compute **vector–Jacobian products** instead.

Vector–Jacobian product = backprop

D

If $y = f(x)$ and $\mathcal{L} = \mathcal{L}(y)$:

$$\underbrace{\overline{x}}_{\frac{\partial \mathcal{L}}{\partial x}} = J^\top \underbrace{\overline{y}}_{\frac{\partial \mathcal{L}}{\partial y}}$$

one VJP per layer — no matrix ever formed

Backprop is a chain of VJPs

For $x \to h_1 \to h_2 \to \mathcal{L}$:

$$\nabla_x \mathcal{L} = J_{h_1,x}^\top \, J_{h_2,h_1}^\top \, \nabla_{h_2} \mathcal{L}$$

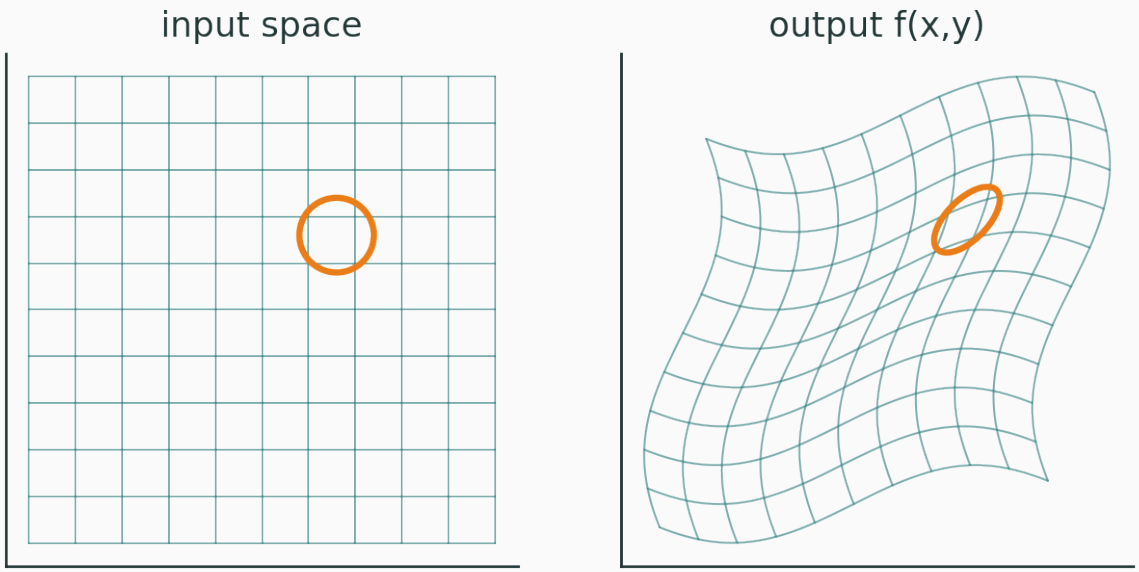
Evaluate **right to left**: $\overline{h}_2 \to \overline{h}_1 \to \overline{x}$ — vectors only.

Interactive: the Jacobian as a local warp

I

INTERACTIVE

(to build) — a 2-D map warps a grid; the local Jacobian sends a small circle to an ellipse.



Where each object shows up in deep learning

Derivative → activations

Scalar activation derivatives drive backprop through nonlinearities:

$$\frac{\mathrm{d}}{\mathrm{d}z}\sigma(z) = \sigma(z)(1-\sigma(z)), \quad \mathrm{ReLU}'(z) = \mathbb{1}[z > 0]$$

Gradient → learning & attribution

- parameter updates: $\theta \leftarrow \theta - \eta \, \nabla_\theta \mathcal{L}$;
- saliency maps: $\nabla_x f_y(x)$;
- adversarial examples: $x_{\text{adv}} = x + \varepsilon \, \text{sign}(\nabla_x \mathcal{L})$.

Jacobian → sensitivity & flows

optional

Appears in backprop, sensitivity analysis, normalizing flows ($\log|\det J|$), neural ODEs, implicit layers — but almost always through Jv or $J^\top v$.

Hessian → curvature & second-order

optional

Newton's method $x_{t+1} = x_t - H^{-1}\nabla f$, Laplace approximation, sharpness/flatness, influence functions — but the full H is usually too large.

Why DL stays first-order

For P parameters, $\nabla_\theta \mathcal{L} \in \mathbb{R}^P$ but $H \in \mathbb{R}^{P \times P}$.

$$P = 10^8 \Longrightarrow H \textbf{ has } 10^{16} \textbf{ entries}$$

So practice uses SGD, momentum, Adam, and diagonal / Hessian-vector approximations.

The autodiff connection

Autodiff is none of the naive options

not symbolic differentiation

not finite differences

Automatic differentiation applies the chain rule to the **executed program** — exact, and cheap.

Scalar loss → reverse mode

Training maps $\theta \in \mathbb{R}^P \to \mathcal{L} \in \mathbb{R}$ (many inputs, one output).

reverse-mode autodiff computes $\nabla_\theta \mathcal{L}$ in one forward-pass cost

Non-scalar output needs a seed vector

D

If $y = f(x) \in \mathbb{R}^m$, `backward()` must know **which** scalar to differentiate — you supply a vector v and it returns

$$v^\top J.$$

This is why PyTorch errors on `.backward()` of a non-scalar tensor without a `gradient=` argument.

PyTorch: Jacobian and VJP

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
def f(x):
    return torch.stack([x[0]**2 + x[1], torch.sin(x[0]*x[1])])

J = torch.autograd.functional.jacobian(f, x)   # full 2x2 Jacobian
v = torch.tensor([1.0, 3.0])
f(x).backward(v)                               # x.grad == v^T J  (a VJP)
```

PyTorch: the Hessian

```
def g(x):
    return x[0]**2 + 3*x[1]**2 + x[0]*x[1]

H = torch.autograd.functional.hessian(g, x)     # -> [[2, 1], [1, 6]]
```

$H = \begin{pmatrix} 2 & 1 \\ 1 & 6 \end{pmatrix}$ — matches the second partials by hand.

Summary

One table to remember

Object	Shape	Main DL use
$f'(x)$	scalar	activation derivative
∇f	d	parameter update
J	$m \times n$	vector-map sensitivity
$J^\top v$	n	backpropagation
H	$d \times d$	curvature
Hv	d	second-order methods

Final mental model — one idea

derivative = slope

gradient = direction of steepest ascent

Jacobian = local linear map

Hessian = curvature

backprop = an efficient chain of vector–Jacobian products

We can now read every derivative object in deep learning.

Next: **computation graphs and backpropagation** — the chain rule, run backward through the program.